SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Concept Mapping (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand	5
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand	5
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand	5
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand	5
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand	5
6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand	5
7	Create a visual concept map for the DBMS software architecture,	BL2 – Understand	5

	illustrating the flow from user request to data retrieval.		
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand	**5**
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand	**5**
10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember	**5**

Code of Conduct:

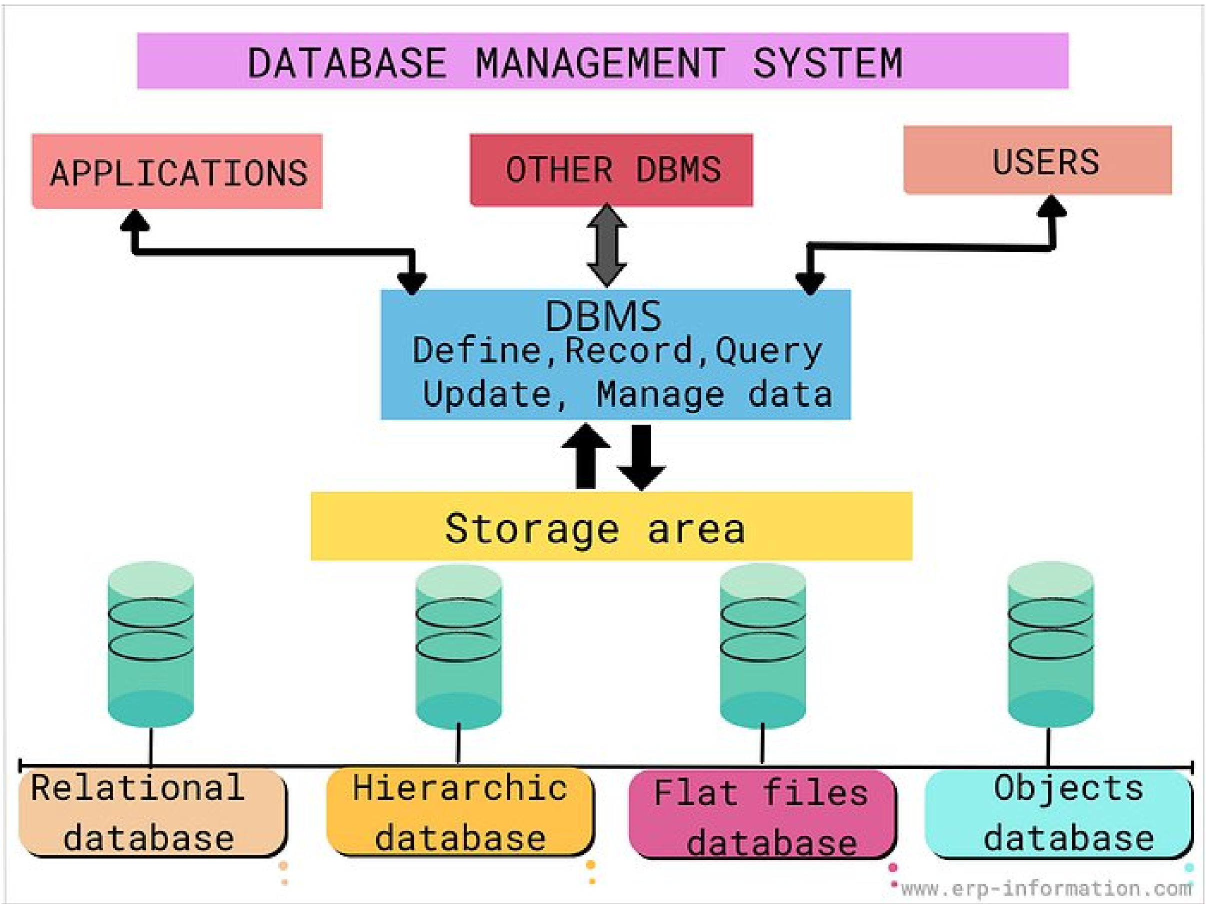
I ____________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show	Shows some integration of literature	Limited integration; mostly isolated	No integration of literature evidence

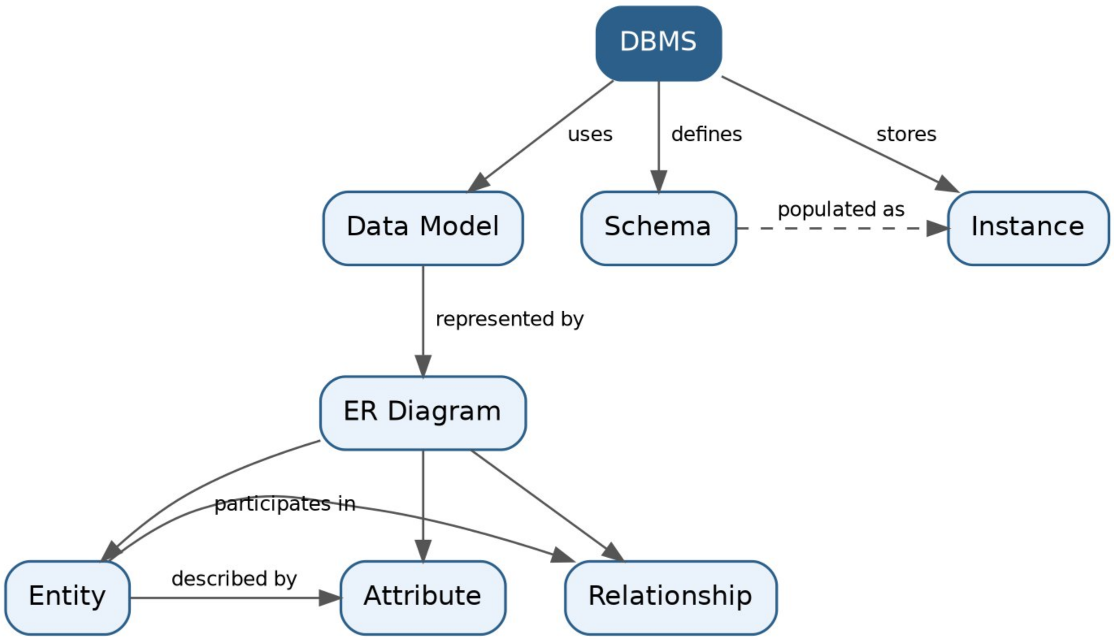
		connections and comparisons		concepts	
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand



Q1. Concept map linking DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

A DBMS (Database Management System) is software that uses an underlying Data Model to define how data is organized. This organization is expressed as a Schema — the fixed structural description of the database (tables, fields, constraints) — while an Instance is the actual data stored at a given moment, which keeps changing as records are inserted, updated, or deleted.

The Data Model is commonly represented visually through an ER Diagram (Entity-Relationship Diagram). An ER Diagram depicts three core building blocks: an Entity (a real-world object such as STUDENT or COURSE), its Attributes (properties describing the entity, e.g., Name, Roll_No), and Relationships (associations between two or more entities, e.g., STUDENT enrolls in COURSE).

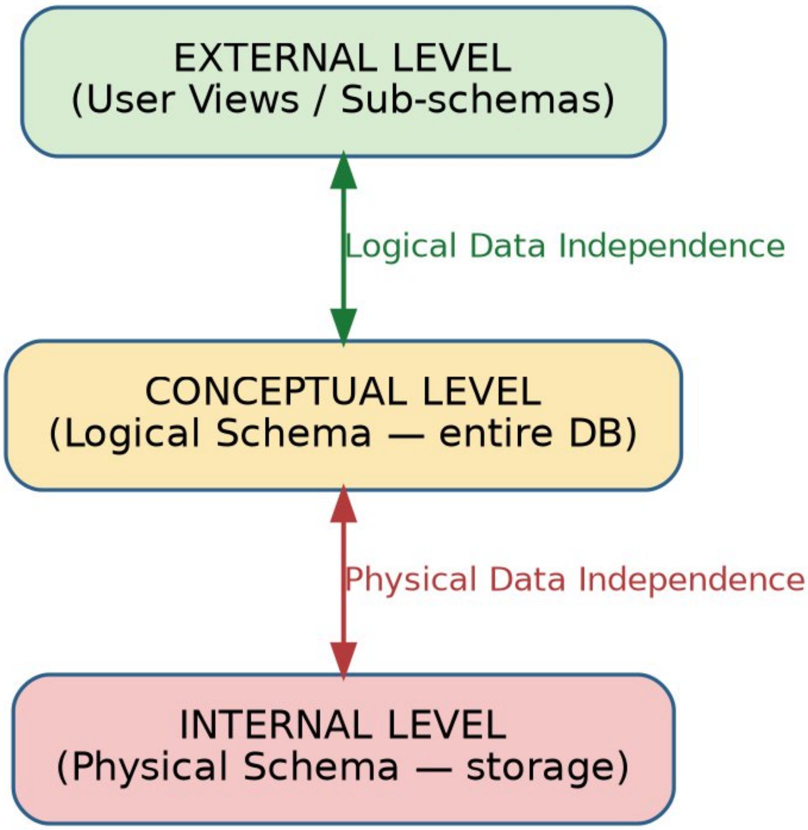
Key Terms Table

Term	Definition
DBMS	Software system that enables creation, storage, retrieval, and management of data in databases.
Schema	The overall logical/physical design or blueprint of the database; changes rarely.
Instance	The actual data present in the database at a specific point in time; changes frequently.
Data Model	A conceptual set of tools/notations for describing data, relationships, and constraints (e.g., Relational, ER, Object-oriented).
ER Diagram	Graphical representation of entities, attributes, and relationships using the ER data model.
Entity	A real-world object or concept that can be distinctly identified (e.g., Employee, Product).
Attribute	A property or characteristic of an entity (e.g., Name, Age, Salary).
Relationship	An association among two or more entities (e.g., Works_For, Enrolls_In).

Q2. Concept map for the three-level ANSI/SPARC architecture showing data independence at each level

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

The ANSI/SPARC architecture divides a database system into three levels to separate what users see from how data is physically stored:

- External Level (View Level): Describes how individual users see their part of the database through views, tailored to each user group's needs.
- Conceptual Level (Logical Level): Describes the structure of the whole database for the community of users — entities, relationships, constraints — independent of storage details.
- Internal Level (Physical Level): Describes how data is actually stored on disk — file organization, indexing, access paths.

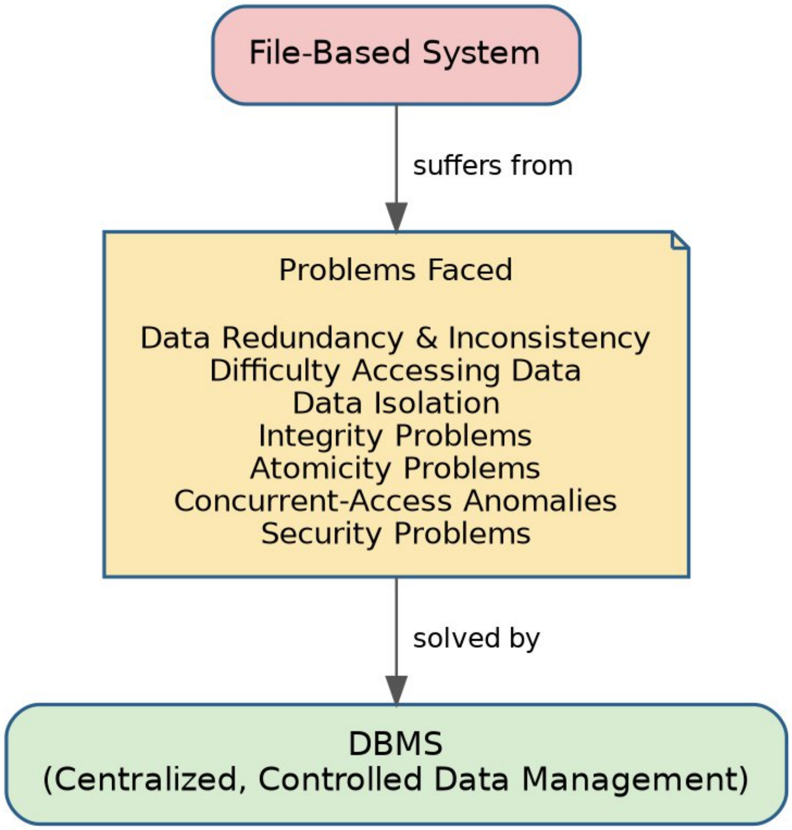
Two forms of data independence connect these levels:

Independence Type	Description
Logical Data Independence	Ability to change the conceptual schema without altering external schemas/application programs (between External and Conceptual levels).
Physical Data Independence	Ability to change the internal (physical/storage) schema without altering the conceptual schema (between Conceptual and Internal levels).

Q3. Relationship between file-based systems and database systems, highlighting problems solved by DBMS

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

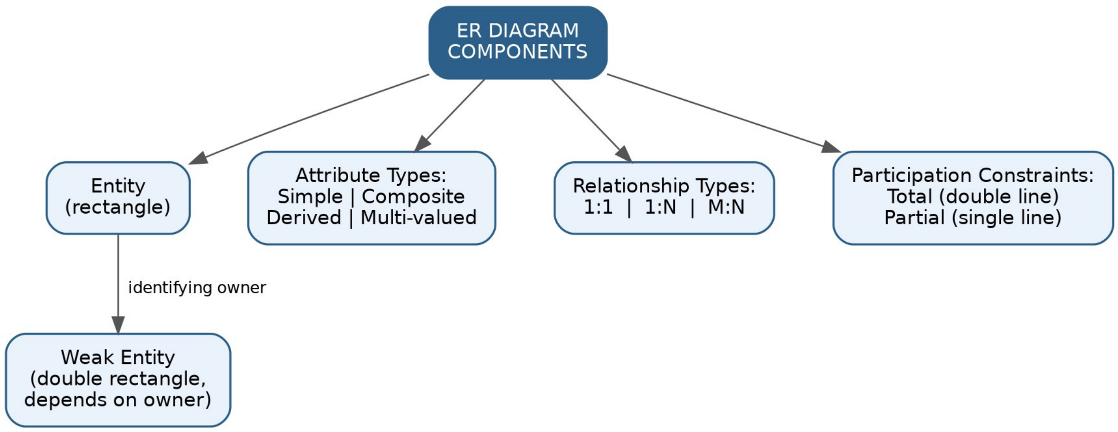
Before DBMS, organizations stored data in File-Based Systems where each application maintained its own separate files. This approach suffered from several drawbacks that the DBMS approach was designed to solve, by centralizing data and providing controlled, shared access.

Problem in File System	How DBMS Solves It	Example
Data Redundancy & Inconsistency	Centralized data storage; single copy of data	Student address stored once, not in every department's file
Difficulty in Accessing Data	Query languages (SQL) allow flexible retrieval	Ad-hoc queries without writing new programs
Data Isolation	Data integrated in one repository, accessible across programs	Common database for Library, Fees, Exam modules
Integrity Problems	Constraints (PK, FK, Check) enforced by DBMS	Age > 0 constraint enforced automatically
Atomicity Problems	Transaction management (ACID properties)	Bank transfer either fully completes or fully rolls back
Concurrent Access Anomalies	Concurrency control (locking, timestamps)	Two users updating same seat booking safely
Security Problems	Authorization and access control	Only HR can view salary data

Q4. Concept map for ER diagram components: entity, weak entity, attribute types, relationship types, participation constraints

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

- Entity: A distinguishable real-world object, represented as a rectangle (e.g., EMPLOYEE).
- Weak Entity: An entity that cannot be uniquely identified by its own attributes alone and depends on an owner/identifying entity (e.g., DEPENDENT depends on EMPLOYEE); shown with a double rectangle.
- Attribute Types: Simple (atomic, e.g., Age), Composite (divisible, e.g., Name = First+Last), Derived (computed, e.g., Age from DOB), Multi-valued (multiple values, e.g., Phone_Numbers).
- Relationship Types: Classified by cardinality — 1:1, 1:N, M:N — describing how many instances of one entity relate to another.
- Participation Constraints: Total participation (every entity instance must participate, shown by double line) vs Partial participation (participation is optional, shown by single line).

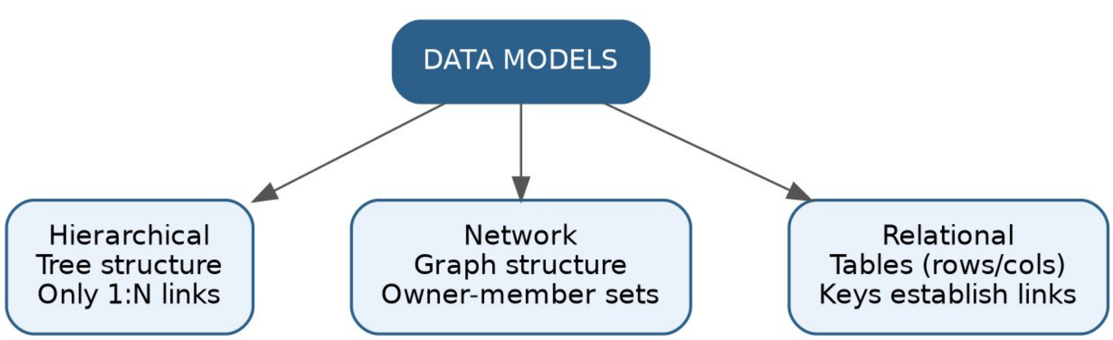
Notation Summary Table

Component	Symbol	Example
Entity	Rectangle	STUDENT
Weak Entity	Double Rectangle	DEPENDENT
Attribute	Oval	Name, Age
Multi-valued Attribute	Double Oval	Phone_Numbers
Relationship	Diamond	ENROLLS_IN
Total Participation	Double Line	Every ORDER must have a CUSTOMER

Q5. Concept map comparing Hierarchical, Network, and Relational data models with key properties

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

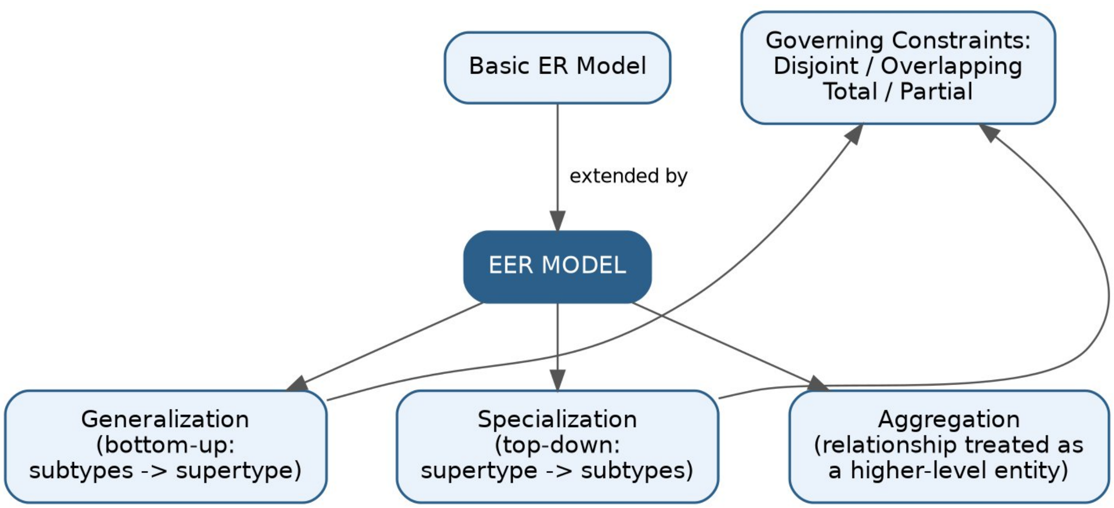
Early data models organized data using explicit navigational structures, while the relational model (introduced by E. F. Codd, 1970) organizes data into simple tables and lets users query using declarative languages like SQL.

Property	Hierarchical	Network	Relational
Structure	Tree (parent-child)	Graph (owner-member sets)	Tables (rows & columns)
Relationship Type	Only 1:N	1:1, 1:N, M:N	1:1, 1:N, M:N (via keys)
Navigation	Top-down, rigid path	Multiple paths via pointers	Declarative (SQL), no navigation needed
Flexibility	Low	Moderate	High
Example System	IMS (IBM)	IDMS	Oracle, MySQL, PostgreSQL

Q6. Concept map for EER model: generalization, specialization, aggregation

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

The Enhanced/Extended ER (EER) Model adds extra semantic constructs to the basic ER model to model complex applications more precisely:

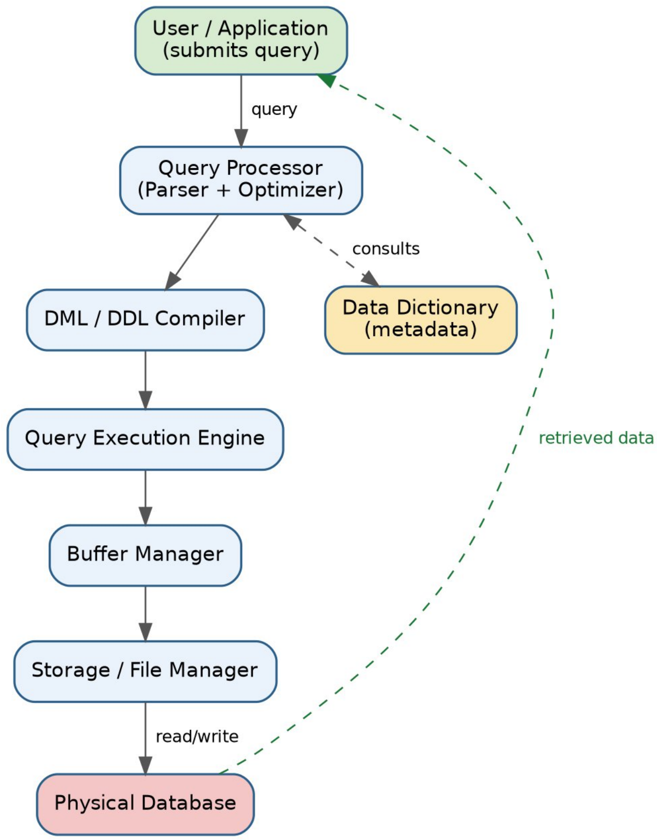
- Generalization: A bottom-up process of combining several entity types that share common attributes into one higher-level generalized entity type (e.g., CAR and TRUCK generalized into VEHICLE).
- Specialization: A top-down process of defining subtypes (subclasses) of an entity type that have distinct attributes (e.g., EMPLOYEE specialized into ENGINEER and MANAGER).
- Aggregation: A concept where a relationship itself is treated as a higher-level entity so it can participate in another relationship (e.g., the relationship WORKS_ON between EMPLOYEE and PROJECT is aggregated so it can relate to MANAGER as its supervisor).

Both generalization and specialization are further governed by disjointness constraints (disjoint/overlapping) and completeness constraints (total/partial).

Q7. Visual concept map for DBMS software architecture, illustrating flow from user request to data retrieval

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

When a user or application submits a query, it passes through several DBMS components before data is returned:

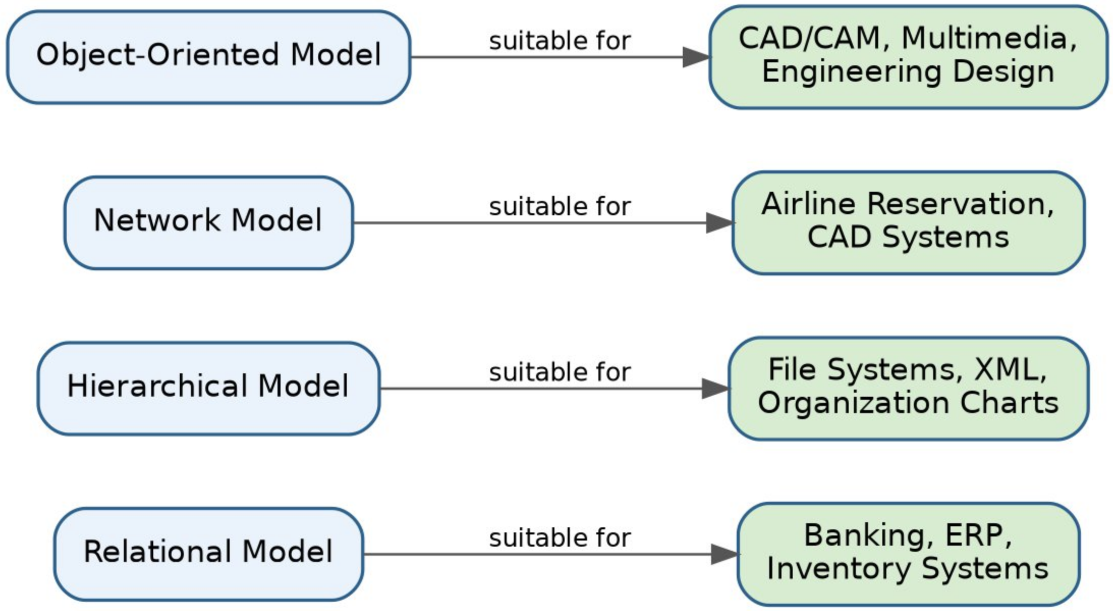
- Query Processor: Parses the query for syntax and semantics, and the Optimizer chooses the most efficient execution plan.
- DML/DDL Compiler: Translates data manipulation or data definition statements into low-level instructions.
- Data Dictionary: Metadata repository consulted for schema information, constraints, and access rights.
- Query Execution Engine: Executes the optimized plan by invoking the storage manager.
- Buffer Manager: Manages the transfer of data between disk and main memory.
- Storage/File Manager: Interfaces with the operating system to read/write data blocks from the physical database on disk.

The result flows back up through the buffer manager and query processor to the user as a result set.

Q8. Map data models from CO1 to real-world applications and explain suitability of each model

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



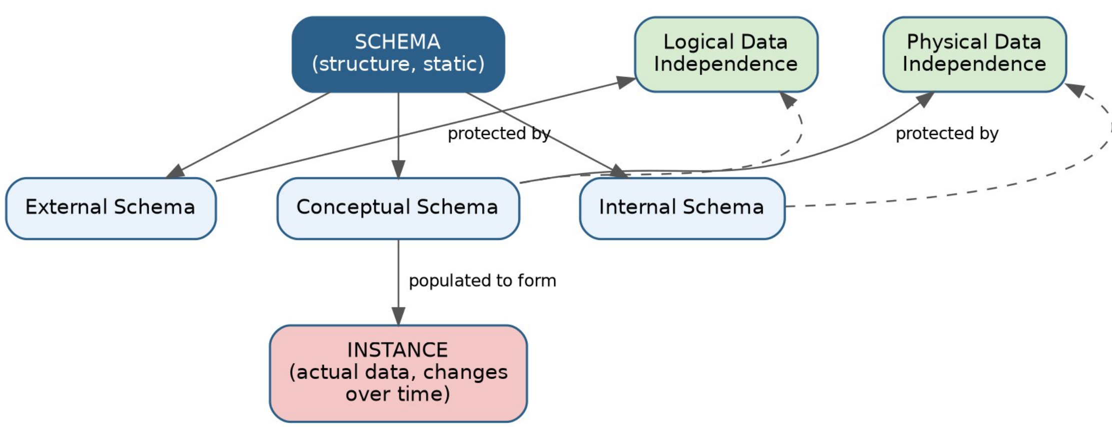
Theory / Suitability

Data Model	Real-World Application	Why Suitable
Relational Model	Banking, ERP, Inventory Management	Simple tabular structure, strong integrity/consistency support, mature SQL tooling for transactional systems.
Hierarchical Model	File systems, XML documents, Org charts	Naturally represents strict parent-child, one-to-many hierarchies.
Network Model	Airline reservation systems, CAD systems	Handles complex many-to-many relationships via owner-member sets efficiently.
Object-Oriented Model	CAD/CAM, multimedia databases, engineering design	Supports complex objects, inheritance, and encapsulation needed for rich data types.

Q9. Concept map for schemas and instances, showing relation to physical and logical data independence

Bloom's Level: BL2 – Understand | Marks: 5

Concept Map



Theory

A Schema is the overall design/blueprint of the database and is largely static; it exists at three sub-levels — External, Conceptual, and Internal — corresponding to the ANSI/SPARC architecture. An Instance is the actual set of data (a snapshot) held in the database at a given moment, and it changes continuously as transactions occur, even though the schema stays constant.

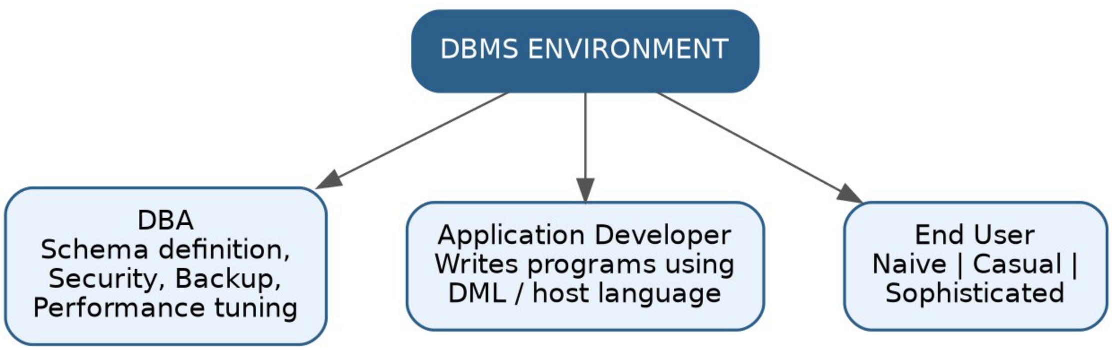
Data independence is the capacity to modify a schema at one level without requiring changes to the schema at the next higher level:

Concept	Explanation
Logical Data Independence	Changes to the Conceptual Schema (e.g., adding a new attribute) do not require changing External Schemas or application programs.
Physical Data Independence	Changes to the Internal Schema (e.g., changing file organization, adding an index) do not require changing the Conceptual Schema.

Q10. Concept map for roles and responsibilities in a DBMS environment: DBA, End User, Application Developer

Bloom's Level: BL1 – Remember | Marks: 5

Concept Map



Theory

Role	Responsibilities
Database Administrator (DBA)	Schema definition and modification, granting authorization/access control, defining storage structure & access methods, routine maintenance, backup & recovery, monitoring performance.
Application Developer / Programmer	Writes application programs in host languages combined with DML (embedded SQL, APIs) to interact with the database and satisfy business requirements.
End User	Interacts with the database through queries or application interfaces without technical DBMS knowledge; classified as: Naive Users (use fixed menu-driven applications), Casual Users (occasionally query using SQL), and Sophisticated Users (write complex queries/analytics).